Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Designing the Flutter Web Client-Side Interface for Your Own Project

Flutter Web Client-Side Design

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 6
Output	Flutter UI-flow diagram, responsive wireframes/prototype, widget and validation plan, and design review
Maximum Score	80 points

Student / Project Information

Block: 3A	Date: October 06, 2026
System Title: Humanitarian Aid Distribution Management System with Machine Learning-Based Household Vulnerability Classification	Instructor: Jean Y. Villasin, MIT
Members: Steven P. Payapaya Kyle M. Borja	

1. Activity Alignment and Purpose

Topic Learning Outcome: Design the Flutter Web user interface, navigation flow, validation plan, and responsive behavior for the semester system.

Course Outcome Alignment: Supports semester-project client-side design by converting approved requirements and use cases into a usable, accessible, responsive, and implementation-ready Flutter Web interface plan.

Syllabus Assessment: Flutter UI flow/wireframe; design review; online interface-design exercise.

Purpose of the Activity

Activity 5 established the requirements baseline and system-specific use case diagram. In Activity 6, your team will translate those approved user goals and requirements into the actual client-side experience of your semester system. You will plan the screen sequence, navigation and routes, responsive layouts, reusable Flutter widgets/components, forms and validation behavior, accessibility and usability decisions, and the loading, empty, success, and error states that users will encounter. The final UI-flow and responsive wireframes will serve as the implementation blueprint for the Flutter Web client in the succeeding development activities.

Important distinction: A wireframe defines structure, hierarchy, content placement, and interaction flow; a prototype simulates interaction; the final Flutter interface is the implemented product. Client-side validation improves usability but does not replace validation and authorization in the REST API.

2. Intended Learning Objectives

- Translate the approved SRS, system scope, user roles, and use cases into a coherent Flutter Web screen flow.
- Identify appropriate widget-based UI structures and distinguish presentational/static components from components that require local or shared state.
- Plan navigation, named routes or route patterns, protected screens, back/exit behavior, and user-role access to major screens.
- Create responsive wireframes that adapt intentionally across representative mobile, tablet, and desktop web widths.
- Design reusable interface components and a consistent visual system for forms, cards, navigation, lists, dialogs, feedback, and common actions.
- Define client-side form validation and loading, empty, success, and error states for important user workflows.
- Review the design for accessibility, usability, requirement traceability, and implementation readiness before Flutter coding begins.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; every member must contribute to flow analysis, wireframing, review, and oral explanation.
Suggested Time	Approximately 3 laboratory hours plus the scheduled design review and online interface-design exercise.
Primary Output	One system-specific Flutter UI-flow diagram and a responsive wireframe/prototype set based on the approved project requirements.
Supporting Output	Screen inventory; widget/state plan; route map; reusable-component plan; form-validation plan; UI-state matrix; accessibility/usability checklist; design-review revision log.
Recommended Tools	Figma, Penpot, diagrams.net/Draw.io, Canva, PowerPoint, Flutter/Dart for optional clickable prototype work, or an equivalent instructor-approved design tool.
Design Review	5–7 minutes per group followed by 2–3 minutes of questions. Every member must be able to explain assigned screens, routes, responsive decisions, and interface states.

4. Pre-Laboratory Requirements

- Approved or revised SRS, user roles/permissions, scope, requirements-to-feature matrix, and system-specific UML use case diagram from Laboratory Activity No. 5.
- Prescribed Flutter Web-REST API-PostgreSQL technology-stack blueprint from Laboratory Activity No. 3 and current architecture diagram from Laboratory Activity No. 2.
- Selected development methodology and semester milestones from Laboratory Activity No. 4.
- List of Must/High-priority features and the user workflows that must be demonstrated during the semester project.
- Any existing branding, content, icons, forms, data samples, or interface references relevant to the team's actual system.
- A shared design workspace or project repository where the UI-flow, wireframes, screenshots, and review notes can be versioned.

5. Part A – Translate Requirements and Use Cases into the UI Flow

Start with the approved requirements and use case diagram—not with visual styling. For each important user goal, identify where the user enters the interface, which screens or overlays are required, where decisions or validation occur, and what successful completion looks like.

Use Case / Req. ID	Actor / Role	Entry Point	Required Screen Sequence	Decision / Validation Point	Expected Destination
FR-001 / Login	LGU Administrator / LGU Staff	Login page	Login → Enter Username & Password → Submit	Required fields; invalid credentials	Dashboard
FR-002 / UC-01	LGU Staff	Dashboard	Dashboard → Household Registration → Household Form → Submit	Required demographic fields; valid input	Household Details
FR-003 / UC-02	LGU Staff	Household Details	Household Details → Assessment Form → Submit Assessment	Required assessment answers	Vulnerability Result
FR-004 / UC-02	LGU Staff	Assessment Form	Submit Assessment → ML Processing → Vulnerability Result	ML response must be Low, Medium, or High	Vulnerability Result
FR-005 / UC-03	LGU Staff	Household Details	Household Details → Aid Distribution → Distribution Form → Submit	Required aid information; duplicate distribution check	Distribution Confirmation
UC-04	LGU Administrator	Dashboard	Dashboard → Reports → Select Date/Filter → Generate	Valid filter/date range	Distribution Report
FR-007 / Access Control	All authenticated users	Any protected route	Login → Protected Screen	Role verification	Authorized screen / Access Denied

Traceability Rule

Every Must/High-priority user-facing requirement should lead to one or more identifiable screens, actions, or interface states. If the wireframes show a major feature that is not supported by the approved requirements, flag it for requirements review before implementation.

6. Part B – Screen Inventory and Widget-Based UI Planning

Create a complete screen inventory before polishing individual wireframes. Use actual project screen names. The "widget/state plan" is a design annotation—not a requirement to write the final Dart code during this activity.

Screen ID / Name	Purpose / Main User	Major UI Sections / Candidate Flutter Widgets	Reusable Component Candidates	State Needed	Data / API Dependency
S-01 Login	Authenticate LGU users	Scaffold, Form, TextFormField, ElevatedButton	AppButton, FieldLabel	Form state, loading	FastAPI authentication
S-02 Dashboard	Overview of system information	AppBar, NavigationRail, Cards, Row, GridView	SummaryCard, StatusChip	Loaded dashboard data	FastAPI
S-03 Household List	View registered households	SearchField, ListView/DataTable, buttons	HouseholdRow, StatusChip	Search/filter state	PostgreSQL through API
S-04 Household Registration	Register household	Form, TextFormField, DropdownButtonFormField	FormField, AppButton	Local form state	FastAPI/PostgreSQL
S-05 Household Details	View household information	Card, Column, ListView, buttons	InfoCard, StatusChip	Loaded record	FastAPI/PostgreSQL
S-06 Assessment Form	Collect vulnerability information	Form, TextFormField, DropdownButtonFormField	FormField, AppButton	Assessment form state	FastAPI
S-07 Vulnerability Result	Display ML classification	Card, StatusChip, Column	VulnerabilityCard	Loading/result state	External ML service through FastAPI
S-08 Aid Distribution	Record relief distribution	Form, Dropdown, TextFormField, Date input	AidItemRow, AppButton	Form/submission state	FastAPI/PostgreSQL
S-09 Distribution History	View previous distributions	ListView/DataTable, filters	DistributionRow	Filter/loading state	FastAPI/PostgreSQL
S-10 Reports	View reports and summaries	Cards, filters, tables	ReportCard, StatusChip	Filter/loading state	FastAPI/PostgreSQL
S-11 User Management	Manage system users	DataTable, Form, Dialog	UserRow, AppButton	Dialog/form state	FastAPI/PostgreSQL
S-12 Access Denied	Inform unauthorized user	Center, Card, Button	FeedbackPanel	None	Authentication state

Widget and State Classification Guide

Design Question	Likely UI Planning Direction	Examples / Notes
Does the component mainly render information from inputs and have no local mutable interaction state?	Plan it as a presentational/reusable widget; it may later be implemented as a StatelessWidget or equivalent composition.	Header, label, summary card, icon-text row, static empty-state panel.
Does the component own temporary local UI state that changes during interaction?	Identify the local state and event that changes it; a StatefulWidget may be appropriate during implementation.	Password visibility, tab selection, expansion, local form interaction.
Is the state shared across screens or tied to asynchronous application data?	Do not force it into one StatefulWidget. Identify the shared/application-state boundary and planned state-management approach.	Authenticated user, loaded records, filters used across screens, async request state.

Can the same structure or behavior appear in several screens?	Extract it as a reusable component with clearly defined inputs, callbacks, and states.	Buttons, form fields, cards, status chips, confirmation dialogs, table/list rows.

Possible Flutter building blocks to annotate when appropriate: Scaffold, AppBar, NavigationRail/Drawer, Row, Column, Wrap, GridView, ListView, Card, Form, TextFormField, DropdownButton/FormField, buttons, dialogs, LayoutBuilder, MediaQuery, and project-specific reusable widgets.

7. Part C – Navigation and Routing Plan

Plan how users move between screens and what should happen when they use browser navigation, direct URLs, sign in/out, role-restricted routes, or invalid routes. The final route structure must match the approved user roles and scope.

Route / Path	Screen / View	Allowed Role(s)	Entered From	Primary Exit / Back Behavior	Guard / Condition
/ login	Login	All users	Application start	Login → Dashboard	Public
/ dashboard	Dashboard	Admin, Staff	Login	Navigation menu	JWT required
/ households	Household List	Admin, Staff	Dashboard	Back → Dashboard	JWT required
/ households/new	Register Household	Staff	Household List	Cancel → Household List	Staff role
/ households/:id	Household Details	Admin, Staff	Household List	Back → Household List	Authorized household access
/ households/:id/assessment	Assessment Form	Staff	Household Details	Cancel → Household Details	Staff role
/households/:id/vulnerability	Vulnerability Result	Staff, Admin	Assessment	Back → Household Details	Authenticated
/distributions/new	Aid Distribution	Staff	Household Details	Cancel → Household Details	Staff role
/distributions	Distribution History	Admin, Staff	Dashboard	Back → Dashboard	JWT required
/reports	Reports	Admin	Dashboard	Back → Dashboard	Admin role
/users	User Management	Admin	Dashboard	Back → Dashboard	Admin role
/access-denied	Access Denied	Authenticated users	Unauthorized route	Back → Previous authorized screen	Role restriction
/404	Invalid Route	All	Invalid URL	Go to Login/Dashboard	Invalid route

- Show the normal route sequence for at least one end-to-end critical workflow.
- Mark routes that require authentication, a specific role, ownership of a record, or another access condition.
- Define what the interface should do for an unknown/invalid route and after session expiration.
- Design navigation so that browser back/forward behavior and direct URLs do not create confusing or unauthorized states.

8. Part D – Responsive Layout and Wireframe Planning

Flutter Web must respond to available space rather than merely shrink a desktop canvas. Use representative widths to test the layout. The exact breakpoints may differ by project, but the team must demonstrate intentional adaptation.

Screen / Module	Narrow / Mobile Behavior	Medium / Tablet Behavior	Wide / Desktop Behavior	Overflow / Long-Content Strategy
Login	Centered single-column form	Centered card	Two-column layout with branding and form	Vertical scrolling
Dashboard	Cards stacked vertically; Drawer navigation	2-column cards; NavigationRail	Sidebar + multi-column dashboard	Cards wrap
Household List	ListView with compact rows	List/table with fewer columns	Full DataTable	Horizontal table scroll only when necessary
Household Form	Single-column fields	Two-column fields	Multi-column organized form	Vertical scrolling
Assessment Form	One question/field per	Grouped sections	Grouped multi-column	Vertical scrolling

	row		sections	
Vulnerability Result	Large result card	Result + household summary	Result + details + action panel	Card wrapping
Aid Distribution	Stacked form	Two-column form	Multi-column form	Vertical scrolling
Reports	Cards and simplified list	Table with filters	Full table and summary cards	Table scrolling
User Management	List-based layout	Compact table	Full DataTable	Horizontal scrolling for table

Minimum Responsive Wireframe Requirement

Produce wireframes for all major screens at the team's primary desktop/web width. In addition, show at least three critical screens at three representative widths (for example, approximately 390 px, 768 px, and 1440 px) so the reviewer can see how navigation, columns, cards, forms, tables/lists, spacing, and actions adapt. Do not rely on tiny text or uncontrolled horizontal overflow as the responsive solution.

9. Part E – Reusable Components and Visual Consistency Plan

Identify the recurring pieces of the interface before implementation. Reusable components should reduce duplicated UI logic while preserving consistent behavior and visual language across the system.

Component / Pattern	Purpose	Used in Screens	Inputs / Variants	Interactive State / Callback	Accessibility / Usability Note
Primary action button	Main system actions	All forms	Save, Submit, Login	Normal, disabled, loading	Clear text label
Form field pattern	Consistent input field	Registration, Assessment, Distribution	Text, number, dropdown	Focus, error, valid	Visible label
Navigation item	Main navigation	Dashboard and all protected pages	Icon + label	Selected/unselected	Keyboard accessible
Record card / row	ⅆDisplay household summary	Dashboard, Household List	Household data	Tap/click	Readable information
Status / feedback component	Show success/error/empty	All data screens	Message + action	Retry/close	Clear recovery instruction
Project-specific component	Display records	Households, reports, users	Record data	View action	Readable row labels

Create a simple interface consistency guide covering: typography hierarchy, spacing system, primary/secondary actions, form-field states, color usage, cards/containers, icon conventions, navigation patterns, confirmation behavior, and feedback messages.

10. Part F – Forms and Input Validation Plan

For each important form, define both the visible interaction and the validation behavior. The message shown to the user should explain what must be corrected. Client-side checks are for immediate feedback; the REST API must still validate submitted data and enforce permissions.

Form / Field	Input / Control	Client-Side Rule	When Error Appears	Error / Helper Message	Submit / API Behavior
Login – Username	TextFormField	Required	When the user leaves the field empty or submits the form	“Username is required.”	Send credentials to FastAPI
Login – Password	TextFormField	Required	When the user leaves the field empty or submits the form	“Password is required.”	Submit only when valid
Household – Head of Family	TextFormField	Required	When the field is empty on submit	“Head of family is required.”	Send household data
Household – Household Size	Number field	Must be a positive number	When the value is empty, zero, or negative	“Enter a valid household size.”	Validate then submit
Household – Address	TextFormField	Required	When the field is empty on submit	“Address is required.”	Do not submit until valid
Household – Contact	TextFormField	Valid contact number format	When the entered number has an invalid format	“Enter a valid contact number.”	Do not submit until valid
Assessment – Vulnerability	Dropdown / FormField	Required	When an assessment	“Please complete this assessment item.”	Do not submit incomplete

Indicators			item is not selected during submission		assessment
Aid – Household	Dropdown	Required	When no household is selected on submit	“Select a household.”	Do not submit until selected
Aid – Item	TextFormField	Required	When the aid item is empty on submit	“Aid item is required.”	Do not submit until valid
Aid – Quantity	Number field	Must be greater than 0	When the quantity is empty, zero, or negative	“Quantity must be greater than 0.”	Do not submit until valid
Aid – Distribution Date	Date input	Valid date	When the date is missing or invalid	“Enter a valid distribution date.”	Do not submit until valid
Reports – Start/End Date	Date input	Valid date range	When the end date is earlier than the start date	“End date must not be earlier than start date.”	Do not request report until valid

- Consider required fields, length, format, numeric/date range, cross-field rules, file type/size, duplicate input, and conditional fields when relevant.
- Show disabled/submitting states so users cannot accidentally submit the same action repeatedly.
- Decide where validation is inline, on field exit, on submit, or a combination—then apply the pattern consistently.

11. Part G – Accessibility and Usability Review

Review the wireframes from the perspective of a user who may navigate by keyboard, use assistive technology, have reduced vision, or encounter the system under time pressure or unreliable connectivity.

Review Area	Required Design Evidence	Status / Revision Note
Labels and instructions	Controls and inputs have visible, understandable labels; placeholder text is not the only label.	Planned
Keyboard and focus	Interactive controls have a logical focus order and visible focus state; key workflows do not require a mouse.	Planned
Contrast and non-color cues	Text and controls remain understandable with sufficient contrast; status/error meaning is not conveyed by color alone.	Planned
Text and layout resilience	Important content remains readable when text is enlarged or the viewport becomes narrow.	Planned
Touch/click targets	Buttons, icons, checkboxes, and menu items are large enough and spaced to prevent accidental activation.	Planned
Feedback and recovery	Errors identify the problem and recovery action; destructive actions use suitable confirmation/undo patterns.	Planned
Semantics / meaningful content	Images/icons and page structure have meaningful labels/semantics where they communicate information.	Planned
Consistency and cognitive load	Navigation, actions, terminology, and form behavior stay consistent across screens.	Planned

12. Part H – Loading, Empty, Success, and Error-State Design

Do not design only the ideal "happy path." Every major data-driven screen or submit action should have a planned state for waiting, no data, completion, and failure.

Screen / Action	Loading State	Empty / No-Data State	Success State	Error / Recovery State	Access / Offline Note
Login	"Signing in..."	Not applicable	Redirect to Dashboard	"Invalid username or password."	Show connection error
Household List	Loading indicator	"No households registered."	Household list displayed	"Unable to load households." + Retry	Require authentication
Household Registration	"Saving household..."	Not applicable	"Household registered successfully."	Display API error; preserve input	Retry
Assessment	"Submitting assessment..."	Not applicable	Vulnerability result displayed	"Assessment could not be processed." + Retry	Show server/ML unavailable message
Vulnerability Result	"Classifying vulnerability..."	"No classification available."	Low / Medium / High displayed	"Classification service unavailable."	Retry
Aid Distribution	"Saving distribution..."	No applicable data	"Aid distribution recorded."	Duplicate/error message + Retry	Show server unavailable
Distribution History	Loading records	"No distribution records found."	Records displayed	Load error + Retry	Authentication required
Reports	"Generating report..."	"No data available for selected period."	Report displayed	"Report could not be generated." + Retry	Show connection message

State-Design Rule

Each state should answer four questions: What happened? What is the system doing now? What can the user do next? Is the action safe to retry? Loading indicators should not hide indefinitely, empty states should offer a meaningful next step when appropriate, and error states should preserve user work whenever possible.

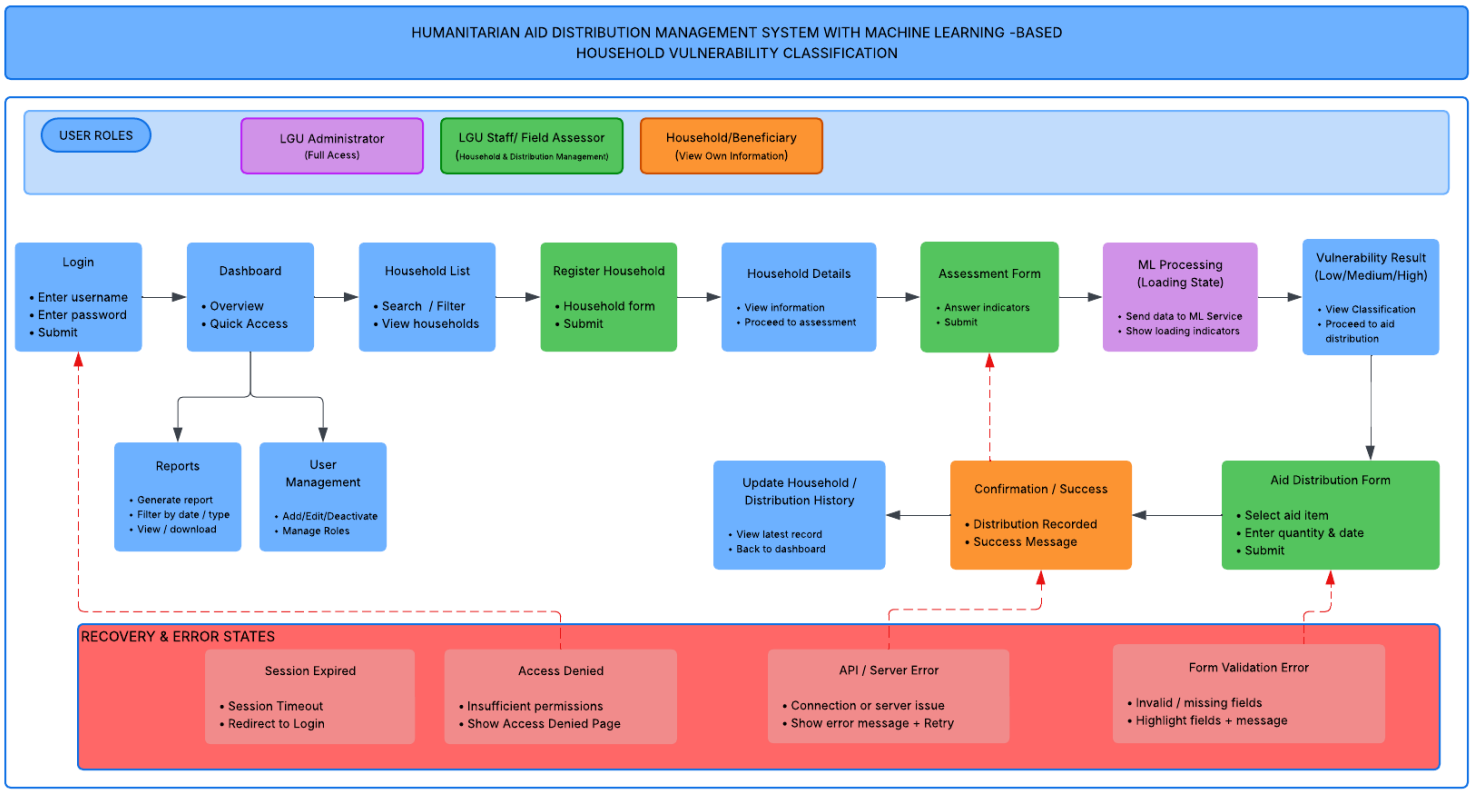
13. Part I – Construct the Flutter UI Flow and Responsive Wireframes

Create one coherent design package that a teammate could use to begin Flutter implementation without guessing the basic navigation, screen hierarchy, responsive behavior, validation, and interface states.

No.	Required Element	What Must Be Shown
1	System identity	Official system title, group, design version/date, and the user role(s) represented in the flow.
2	Screen inventory	Every major user-facing screen required by the approved project scope, using stable screen IDs/names.
3	UI-flow diagram	Directional links between screens/overlays, with labels for important user actions, decisions, and success/error branches.
4	Navigation and routing	Planned route/path or route name, main navigation structure, role restrictions, and important entry/exit behavior.
5	Widget-based structure	Annotated sections/components showing suitable Flutter UI building blocks and the separation of reusable components.
6	State planning	Identify where local interaction state, shared/application state, or asynchronous request state is required.
7	Responsive behavior	Desktop/wide layout plus responsive adaptations; at least three critical screens demonstrated at three representative widths.
8	Forms and validation	Key inputs, controls, validation conditions, error feedback, submit behavior, and disabled/loading states.
9	Accessibility/usability	Labels, focus/keyboard behavior, readable hierarchy, contrast/non-color cues, target sizing, and recovery guidance.
10	System feedback states	Loading, empty, success, and error/retry states for important data screens and transactions.
11	Requirement traceability	Wireframe screens/actions are labeled or cross-referenced to the relevant use cases, requirements, or Must/High-priority features.
12	Design consistency	Common typography, spacing, component behavior, terminology, and action placement are applied consistently across the design.

Minimum UI-Flow Rule

The design package must contain at least one complete critical path from user entry or sign-in -> navigation -> feature screen -> form/detail action -> validation/request state -> success or recoverable error -> updated destination. The flow must be understandable even without opening the Flutter code.



14. Reference Example – Do Not Copy as Your Final Output

The example below shows the level of flow clarity expected. Replace every label, screen, state, action, and route with the actual design of your approved semester system.

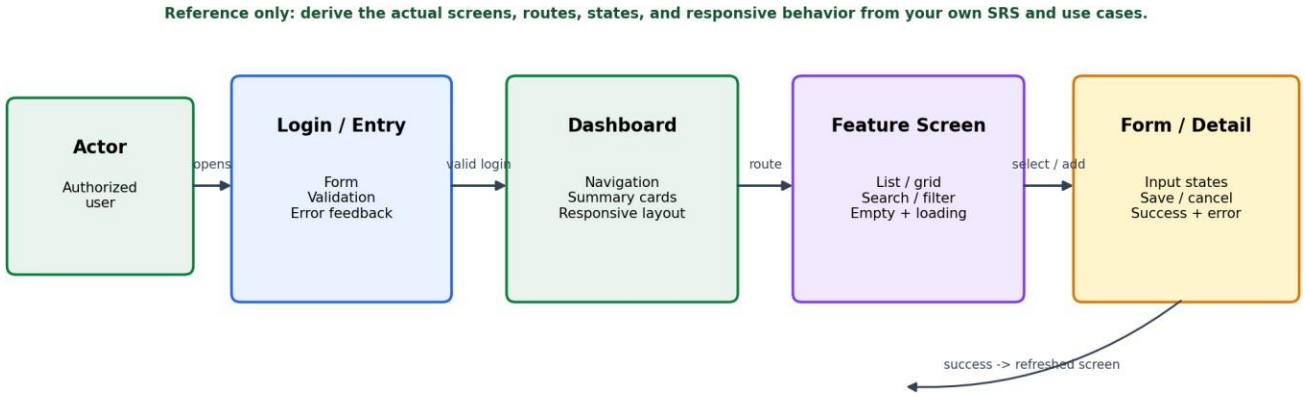


Figure 1. Reference client-side flow showing screens, navigation, validation, and interface states.

15. Recommended Flutter UI Design Annotations

Area	Possible Annotation	What It Communicates
Page shell	Scaffold / responsive shell	Overall page structure, navigation, content area, and global feedback region.
Navigation	NavigationRail / Drawer / menu / tabs	How primary destinations change by viewport and role.
Layout	Row / Column / Wrap / Grid / List	How content groups, wraps, stacks, scrolls, or changes column count.
Form	Form / TextFormField / dropdown / date input	Input grouping, validation feedback, submit behavior, and field states.
Reusable UI	ProjectCard / StatusChip / AppButton / FieldLabel	A project-specific reusable component and its variants.
Responsive logic	LayoutBuilder / MediaQuery / width class	Where structure or component presentation changes because of available space.
Async state	loading / empty / success / error	What the user sees before, during, and after a data request or mutation.

16. Laboratory Procedure

Phase 1 – Review the approved requirements (20 minutes)

Identify the Must/High-priority use cases, user roles, permissions, and major data interactions that require client-side screens.

Phase 2 – Build the screen inventory and UI flow (25 minutes)

List the screens, assign stable screen IDs, connect them using the actual user workflow, and identify decisions, guards, and important feedback states.

Phase 3 – Plan navigation, widgets, and reusable components (30 minutes)

Define the route structure, primary navigation pattern, major widget groups, reusable components, and state boundaries.

Phase 4 – Draft responsive wireframes (50 minutes)

Create the major desktop/web wireframes and the required responsive variants for at least three critical screens.

Phase 5 – Add validation and interface states (20 minutes)

Annotate forms, client-side validation, loading, empty, success, and error/recovery behavior.

Phase 6 – Accessibility and usability review (15 minutes)

Check keyboard/focus behavior, labels, contrast/non-color cues, layout resilience, target sizing, and user recovery.

Phase 7 – Conduct peer/instructor design review (15 minutes)

Use the review log below. Record defects, unclear flows, missing states, requirement mismatches, and responsive problems.

Phase 8 – Revise, export, and submit (5 minutes)

Update the design version, export required files, commit the design artifacts, and submit the required links/files through google classroom.

17. Part J – Design Review and Revision Log

Use this log during the group, peer, or instructor design review. Do not silently fix major findings; record the problem and the revision decision so the team can explain how the design improved.

Issue ID	Reviewer Finding / Question	Affected Screen / Use Case	Priority	Revision Decision / Action	Status
DR-001	Team/ Dashboard contains too much information on mobile.	S-02 Dashboard	High	Stack summary cards vertically on narrow screens.	Resolved
DR-002	Team/ Household form may become too wide on desktop.	S-04 Household Registration	Medium	Use responsive columns with LayoutBuilder.	Resolved
DR-003	Team/ ML classification may take several seconds.	S-07 Vulnerability Result	High	Add loading state and prevent duplicate submission.	Resolved
DR-004	Team/ Users may not know why a submission failed.	All forms	High	Add clear error message and Retry action.	Resolved
DR-005	Team/ Vulnerability status should not depend only on color.	S-07	Medium	Display Low/Medium/High text labels with status indicators.	Resolved

18. Final Quality Checklist

Checklist A	Checklist B
☑ UI flow is derived from the approved SRS and use case diagram.	☑ Major screens are wireframed for the primary desktop/web width.
☑ Every Must/High-priority user-facing workflow maps to identifiable screens/actions.	☑ At least three critical screens show intentional mobile, tablet, and desktop adaptations.
☑ Screen names/IDs and route labels are consistent across the flow and wireframes.	☑ Layouts avoid unreadably small text and uncontrolled horizontal overflow.
☑ Protected or role-specific screens identify the relevant access condition.	☑ Keyboard/focus, labels, contrast/non-color cues, target size, and error recovery are reviewed.
☑ Reusable components are identified instead of duplicating the same pattern across screens.	☑ Terminology, buttons, spacing, navigation, and feedback patterns are consistent.
☑ Forms show required fields, validation behavior, error feedback, and submit/loading state.	☑ The design-review log records findings and the team's revision decisions.
☑ Loading, empty, success, and error/retry states are designed for important data workflows.	☑ Every member can explain assigned screens, routes, state behavior, and responsive choices without reading a script.

19. Submission Requirements

- Final Flutter UI-flow diagram in PDF or PNG format plus the editable source file.
- Responsive wireframe/prototype package showing all major screens and the required three-width variants for at least three critical screens; submit the editable file or accessible share link when the tool is web-based.
- Completed screen/widget inventory, route plan, reusable-component plan, validation plan, UI-state matrix, accessibility/usability review, and design-review revision log.
- Updated project repository or shared workspace under a folder such as docs/ui-design/ containing the current design version and exported review artifacts.
- Repository/share link and required files submitted through MasaoLMS using the naming convention IT110_LA6_SystemName.
- Complete the online interface-design exercise and incorporate instructor feedback that affects the project interface before implementation.

Academic Integrity and Competency Validation

The UI flow and wireframes must represent the team's own approved system, users, requirements, and design decisions. AI or design templates may be used to clarify concepts or accelerate drafting, but the submitted screen flow, labels, responsive choices, validation behavior, accessibility decisions, and state designs must be reviewed and understood by the team. The instructor may ask any member to trace a workflow or justify a screen/component decision without assistance.

20. Scoring Rubric – 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent with the approved requirements, technically weak, visually unclear, or not responsive. Design-review points may be reduced for unequal participation, excessive reading, or inability to explain the team's own interface decisions.

Criterion	Evidence for Full Credit	Points
Requirements-to-UI-flow traceability	The UI flow is complete for the approved scope; Must/High-priority use cases and requirements map clearly to screens, actions, decisions, and expected destinations.	10
Navigation and routing plan	Routes, navigation hierarchy, entry/exit behavior, role restrictions, protected screens, and important browser/back behavior are clear and internally consistent.	10
Responsive wireframes and layout design	Major screens are complete and readable; at least three critical screens demonstrate intentional adaptation across representative mobile, tablet, and desktop widths without uncontrolled overflow.	15
Widget structure and reusable components	The design shows sensible widget-based composition, identifies reusable UI patterns, and distinguishes local interaction state from broader/shared or asynchronous application state.	10
Forms, validation, and interface states	Important forms include usable validation and feedback; loading, empty, success, error/retry, disabled, and submit states are appropriately designed.	10
Accessibility and usability	The design addresses labels, focus/keyboard access, readable hierarchy, contrast/non-color cues, target sizing, consistency, user guidance, and recovery from errors.	8
Visual consistency and design documentation	Screen IDs, routes, terminology, typography, spacing, controls, annotations, and versioned design artifacts are clear, consistent, and implementation-ready.	7
Design review and oral defense	The group uses the UI flow/wireframes effectively, every member participates, revisions are documented, and the team can justify responsive, navigation, component, validation, and state decisions.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

21. Post-Laboratory Reflection

Which user workflow was the most difficult to translate from the SRS/use case diagram into a clear screen flow, and why? - The most difficult workflow was the household assessment and vulnerability classification workflow because it involves several connected steps. The LGU Staff must select a household, complete the assessment, submit the information, wait for the ML classification, and then view the Low, Medium, or High vulnerability result. The interface also needs loading and error states if the ML service is unavailable.

Which interface element should become a reusable Flutter widget/component first, and which screens will use it? - The Primary Action Button should become one of the first reusable Flutter components because many screens require actions such as Login, Save, Submit Assessment, Record Distribution, Generate Report, and Retry. Using one reusable component keeps the buttons consistent across the system.

What was the most important change your team made between narrow/mobile, medium/tablet, and wide/desktop layouts? - The most important change was the navigation and content arrangement. On mobile, the navigation changes to a compact Drawer and information cards are stacked vertically. On tablet, cards are arranged in two columns. On desktop, the system uses a sidebar or NavigationRail with multiple dashboard sections displayed at the same time. This follows the requirement that Flutter Web should intentionally adapt to available space instead of simply shrinking the desktop layout**.**

Which validation, loading, empty, success, or error state would create the greatest usability problem if it were not designed before implementation? - The assessment and ML classification loading/error state would create the greatest usability problem. The user needs to know whether the assessment is still being processed, whether the classification was successfully returned, or whether the ML service failed. Without these states, the LGU Staff might submit the assessment repeatedly or assume that the system is not working.

What specific design issue must your team resolve before beginning the next Flutter client-side implementation activity? - The team must finalize the responsive wireframes, route restrictions, form validation, and API loading/error states before implementation. This will ensure that the Flutter developers know which screens to build, how users navigate between them, which roles can access each screen, and what happens when an API request succeeds or fails.